

Project 0 — Getting Started with Verilator

Designing and Securing Modern Computing Systems
From Logic Gates to Trusted Kernels

Purpose: Get Your Verilog Environment Up & Running

All of the projects in this book will make use of a Verilog development environment for building digital circuits, so this first project is meant to get you started with that.

Step 1: Download Verilator

Your first job is to get a copy of *Verilator*, a Verilog simulator that will run on your computer, provided that you have a compiler. Verilator can be found at the following website:

`https://verilator.org/guide/latest/install.html`

The page may be hosted on ***veripool.org***, so if you see that name in the address bar, don't be surprised or think that the site has been spoofed.

Install the application and get it running.

Step 2: Download the Project-0 Tarball

For testing out your Verilog development environment, we have provided two initial designs that should run right out of the box:

- **Project 0, Combinational Logic (LC0)** — a combinational logic example that implements a 16-bit ALU, performing addition, subtraction, AND, and OR functions.
- **Project 0, Sequential Logic (LS0)** — a sequential logic example that implements a 32-bit counter.

Download the directory tarball from the course website and decompress it. For example, you should find the following files in the **LC0** subdirectory:

- Makefile — contains the rules for building and running the simulation
- **alu16.v** — this is the design file — in most projects, your code goes here (the name will change from project to project)
- sim_main.cpp — code for implementing clocks and interactive behaviors
- top.v — test-harness code written in Verilog

All of the projects in this chapter will have a form similar to this. The **Makefile** allows you to simply type 'make' in the project directory, and it will build and run. The **sim_main.cpp** file is there to make clocks easier to implement. The **top.v** file provides a test harness in Verilog that instantiates and runs whatever design is intended for the project. Lastly, you will have a design file, and the file's name needs to match the name of the module that you design. This design file will contain the circuit that you are to design and build for the project. In this case, it is a 16-bit ALU; in the sequential example it is a 32-bit counter. In general, the only file you will ever need to modify is the design file. That is why it is bolded above.

Step 3: Run the Designs

In this first project, your job is simply to get things up and running. You do not need to do any design, simply run the designs that you have been given. The design in Project LC0 is a 16-bit ALU and has the following form:

```
//
// ALU func:
// 00 = add
// 01 = sub
// 10 = AND
// 11 = OR
//

module alu16 (
    input [1:0] func,
    input [15:0] in1,
    input [15:0] in2,
    output [15:0] out
);

    assign  out = (func == 2'd0) ? in1 + in2  : 16'dZ;
    assign  out = (func == 2'd1) ? in1 - in2  : 16'dZ;
    assign  out = (func == 2'd2) ? in1 & in2  : 16'dZ;
    assign  out = (func == 2'd3) ? in1 | in2  : 16'dZ;

endmodule
```

When you run the combinational example in the LC0 subdirectory, you should see the following output:

```
op 0 (+) in1 00eb (  235) in2 24f1 ( 9457) out 25dc ( 9692) [verify 25dc ( 9692)]
op 1 (-) in1 00eb (  235) in2 24f1 ( 9457) out dbfa (56314) [verify dbfa (56314)]
op 2 (&) in1 00eb (  235) in2 24f1 ( 9457) out 00e1 (  225) [verify 00e1 (  225)]
op 3 (|) in1 00eb (  235) in2 24f1 ( 9457) out 24fb ( 9467) [verify 24fb ( 9467)]
op 0 (+) in1 1376 ( 4982) in2 dedf (57055) out f255 (62037) [verify f255 (62037)]
op 1 (-) in1 1376 ( 4982) in2 dedf (57055) out 3497 (13463) [verify 3497 (13463)]
op 2 (&) in1 1376 ( 4982) in2 dedf (57055) out 1256 ( 4694) [verify 1256 ( 4694)]
op 3 (|) in1 1376 ( 4982) in2 dedf (57055) out dfff (57343) [verify dfff (57343)]
op 0 (+) in1 2601 ( 9729) in2 98cd (39117) out bece (48846) [verify bece (48846)]
op 1 (-) in1 2601 ( 9729) in2 98cd (39117) out 8d34 (36148) [verify 8d34 (36148)]
op 2 (&) in1 2601 ( 9729) in2 98cd (39117) out 0001 (    1) [verify 0001 (    1)]
op 3 (|) in1 2601 ( 9729) in2 98cd (39117) out becd (48845) [verify becd (48845)]
op 0 (+) in1 388c (14476) in2 52bb (21179) out 8b47 (35655) [verify 8b47 (35655)]
op 1 (-) in1 388c (14476) in2 52bb (21179) out e5d1 (58833) [verify e5d1 (58833)]
op 2 (&) in1 388c (14476) in2 52bb (21179) out 1088 ( 4232) [verify 1088 ( 4232)]
op 3 (|) in1 388c (14476) in2 52bb (21179) out 7abf (31423) [verify 7abf (31423)]
op 0 (+) in1 4b17 (19223) in2 0ca9 ( 3241) out 57c0 (22464) [verify 57c0 (22464)]
op 1 (-) in1 4b17 (19223) in2 0ca9 ( 3241) out 3e6e (15982) [verify 3e6e (15982)]
op 2 (&) in1 4b17 (19223) in2 0ca9 ( 3241) out 0801 ( 2049) [verify 0801 ( 2049)]
op 3 (|) in1 4b17 (19223) in2 0ca9 ( 3241) out 4fbf (20415) [verify 4fbf (20415)]
op 0 (+) in1 5da2 (23970) in2 c697 (50839) out 2439 ( 9273) [verify 2439 ( 9273)]
op 1 (-) in1 5da2 (23970) in2 c697 (50839) out 970b (38667) [verify 970b (38667)]
op 2 (&) in1 5da2 (23970) in2 c697 (50839) out 4482 (17538) [verify 4482 (17538)]
op 3 (|) in1 5da2 (23970) in2 c697 (50839) out dfb7 (57271) [verify dfb7 (57271)]
…
```

This cycles through four different operations (0=add, 1=subtract, 2=AND, 3=OR), and for each it tests a set of values that are shown in both hex and decimal forms. Then the ‘verify’ section is produced in the test harness itself, which shows that one can use test harnesses not only to output values but to verify circuit correctness as well.

The design in Project LS0 is a 32-bit counter and has the following form:

```
module counter32 (
    input clk,
    input reset,
    output [31:0] out
);

    reg     [31:0]  counter;
    wire    [31:0]  adder = counter + 1;

    assign out = counter;

    always @ (posedge clk) begin
        counter <= (reset) ? 32'd0 : adder;
    end

endmodule
```

When you run the sequential example in the LS0 subdirectory, you should see the following output:

```
cycle 0001 - res 1 ctr 0
cycle 0002 - res 0 ctr 0
cycle 0003 - res 0 ctr 1
cycle 0004 - res 0 ctr 2
cycle 0005 - res 0 ctr 3
cycle 0006 - res 0 ctr 4
cycle 0007 - res 0 ctr 5
cycle 0008 - res 0 ctr 6
cycle 0009 - res 0 ctr 7
cycle 0010 - res 0 ctr 8
cycle 0011 - res 0 ctr 9
cycle 0012 - res 0 ctr 10
cycle 0013 - res 0 ctr 11
cycle 0014 - res 0 ctr 12
cycle 0015 - res 0 ctr 13
cycle 0016 - res 0 ctr 14
cycle 0017 - res 0 ctr 15
cycle 0018 - res 0 ctr 16
cycle 0019 - res 0 ctr 17
cycle 0020 - res 0 ctr 18
cycle 0021 - res 0 ctr 19
cycle 0022 - res 0 ctr 20
…
```

In the first cycle, the ‘reset’ signal is high, indicating that the circuit should initialize itself. When the ‘reset’ signal is low, the circuit begins counting upward.

Step 4: Run All the Other Designs

Now that you have a design environment, grab the other designs we have shown you, implement them, and run them.